

FT_IRC / DETAIL DESIGN

ft_irc

MODE詳細設計書

i・t・k・o・lの解析と状態変更

設計対象: Mandatory要件のみ

要件参照元: ft_irc subject Version 11.0

文書番号: 05

文書言語: 日本語

目次

1. 文書の目的と対象	3	11. Mode `k`	13
2. 基本方針	4	12. Mode `o`	14
3. MODE Handler	5	13. Mode `l`	15
4. Mode照会	6	14. 未知Mode	16
5. Mode変更の解析結果	7	15. 権限検証	17
6. Mode文字列の解析	8	16. MODE通知	18
7. Mode Parameter消費規則	9	17. 処理例	19
8. 成功変更の集約	10	18. Modeと他Commandの依存	20
9. Mode `i`	11	19. テスト観点	21
10. Mode `t`	12	20. 実装完了条件	22

1. 文書の目的と対象

本書は、Channel MODE CommandとMandatory Mode i、t、k、o、lを実装するための詳細設計を定義する。

形式:

```
MODE <channel>  
MODE <channel> <modes> [mode parameters...]
```

対象:

- ・ +i / -i
- ・ +t / -t
- ・ +k / -k
- ・ +o / -o
- ・ +l / -l
- ・ Mode照会
- ・ 複数Modeの左から右への処理
- ・ 成功した変更の通知
- ・ Mode固有エラー

対象外:

- ・ User Mode
- ・ Ban、Voice、SecretなどMandatory外Mode
- ・ Server間MODE転送

2. 基本方針

Mode照会:

- ・ 登録済みClientなら実行可能
- ・ Channel Memberでなくても照会可能
- ・ Channelが存在しなければ403

Mode変更:

- ・ 実行ClientがChannel Memberであること
- ・ 実行ClientがChannel Operatorであること
- ・ Mode文字列を左から右へ処理すること
- ・ 1文字の失敗でCommand全体を中止しないこと
- ・ 成功した変更だけを1つのMODE通知へまとめること

User Mode形式:

```
MODE <nickname> ...
```

はMandatory対象外である。targetが#で始まらない場合は421を返す。

3. MODE Handler

```
void Server::handleMode(  
    int fd,  
    const Message &message  
);
```

処理分岐:

```
params.size() == 0  
-> 461  
  
params.size() == 1  
-> Channel Mode照会  
  
params.size() >= 2  
-> Channel Mode変更
```

共通検索:

1. params[0]をChannel名として検証する
2. Channelを検索する
3. 存在しなければ403

4. Mode照会

入力:

```
MODE #general
```

応答:

```
:ircserv.local 324 <nick> #general +<modes> [mode parameters]
```

Mode表示順:

```
i  
+t  
+k  
+l
```

実際には有効な文字だけをitkl順で連結する。

例:

```
:ircserv.local 324 alice #general +itkl secret 10
```

oはMemberごとの権限であり、Channel全体のMode照会には含めない。

KeyはOperator以外にも返す設計とする。Mandatory範囲ではKey秘匿要件がないため、状態確認の単純性を優先する。

5. Mode変更の解析結果

内部構造:

```
struct ModeChange
{
    bool    adding;
    char    mode;
    bool    hasArgument;
    std::string argument;
};
```

解析結果を保存するContainer:

```
typedef std::vector<ModeChange> ModeChangeList;
```

実装を簡潔にする場合は、全変更を先に構造化せず、Mode文字列を走査しながら引数Indexを進めてもよい。ただし検証と通知生成の責務は分ける。

6. Mode文字列の解析

入力例:

```
MODE #general +it-kl secret
```

Mode文字列:

```
+it-kl
```

解析状態:

- ・ 現在の符号adding
- ・ Mode Parameterの参照位置argIndex
- ・ 成功したMode文字列
- ・ 成功したMode Parameter一覧

擬似コード:

```
bool adding = true;
bool signSeen = false;
std::size_t argIndex = 2;

for each char c in modeString:
    if c == '+':
        adding = true
        signSeen = true
        continue

    if c == '-':
        adding = false
        signSeen = true
        continue

    if !signSeen:
        send 472
        continue

    process one mode character
```

Mode文字列に符号がない場合、各文字へ472を返し変更しない。

7. Mode Parameter消費規則

Mode	+時	-時
i	不要	不要
t	不要	不要
k	必須	不要
o	必須	必須
l	必須	不要

Parameter不足:

- ・ そのMode文字の変更を行わない
- ・ 461 MODE :Not enough parametersを返す
- ・ 後続Modeの解析は継続する
- ・ 不足したParameterは消費しない

-kにParameterが付いていても、本設計では消費しない。次のParameter必要Modeが使用する。

8. 成功変更の集約

通知用状態:

```
std::string changedModes;  
std::vector<std::string> changedParams;  
char lastSign;
```

成功した変更だけを追加する。

例:

```
入力:  
MODE #general +it-o bob  
  
成功:  
+i  
+t  
-o bob  
  
通知:  
:alice!a@host MODE #general +it-o bob
```

同じ符号が連続する場合は符号を1回だけ付ける。符号が変わる時だけ新しい+または-を追加する。

状態が既に同じで実変更がない場合:

- ・ Errorは返さない
- ・ 通知へ含めない

成功変更が0件ならMODE通知を送らない。

9. Mode `i`

意味:

- ・ +i: Invite-onlyを有効にする
- ・ -i: Invite-onlyを無効にする

API:

```
channel.setInviteOnly(adding);
```

+i有効時のJOIN:

- ・ ChannelのInvite集合にClient FDが必要
- ・ Inviteがなければ473
- ・ JOIN成功後にInviteを削除する

-i:

- ・ 既存Invite集合は保持する
- ・ 将来再び+iになった場合に有効なInviteとして扱う

引数は消費しない。

10. Mode `t`

意味:

- ・ +t: Topic変更をOperatorへ制限
- ・ -t: Channel MemberならTopic変更可能

API:

```
channel.setTopicRestricted(adding);
```

新規Channelでは+tを初期値とする。

TOPIC照会はtに関係なくMemberなら実行できる。

引数は消費しない。

11. Mode `k`

意味:

- ・ `+k <key>`: Channel Keyを設定または置換
- ・ `-k`: Channel Keyを削除

`+k`検証:

- ・ 引数が存在する
- ・ 1文字以上23文字以下
- ・ 空白、NUL、CR、LF、TABを含まない

API:

```
channel.setChannelKey(key);
```

既にKeyがある場合でもOperatorは新しいKeyへ置換できる。467は使用しない。

`-k`:

```
channel.clearChannelKey();
```

通知:

```
MODE #general +k secret  
MODE #general -k
```

JOIN時:

- ・ `hasKey()`がtrueなら指定Keyと完全一致で比較する
- ・ 不一致またはKeyなしなら475

12. Mode `o`

意味:

- ・ `+o <nickname>`: 対象MemberへOperator権限を付与
- ・ `-o <nickname>`: 対象MemberからOperator権限を削除

検証:

1. 引数存在確認
2. NicknameでClient検索
3. Clientが存在しなければ401
4. 対象ClientがChannel Memberでなければ441
5. `+o`ならOperator集合へ追加
6. `-o`ならOperator集合から削除

権限が既に期待状態なら実変更なしとする。

最後のOperatorを`-o`しても許可する。自動で別MemberをOperatorへしない。

通知:

```
:<operatorPrefix> MODE #general +o bob  
:<operatorPrefix> MODE #general -o bob
```

13. Mode `l`

意味:

- ・ `+l <limit>`: Member上限を設定
- ・ `-l`: Member上限を削除

`+l`検証:

- ・ 引数がASCII数字だけ
- ・ 先頭+または-を許可しない
- ・ 0を許可しない
- ・ `std::size_t`へoverflowしない
- ・ 上限値は100000以下

```
channel.setUserLimit(limit);
```

現在Member数より小さいLimitも設定可能とする。既存Memberは削除せず、Member数がLimit未満になるまで新規JOINを拒否する。

`-l`:

```
channel.clearUserLimit();
```

JOIN時:

```
if limitEnabled
  && memberCount >= userLimit:
  471
```

14. 未知Mode

Mandatory外の文字を受信した場合:

```
472 <char> :is unknown mode char to me for <channel>
```

未知Modeは引数を消費しない。

例:

```
MODE #general +im
```

+iは成功し、mへ472を返す。成功した+i通知は送信する。

15. 権限検証

変更Commandの検証順:

1. Channel存在確認
2. 実行ClientのMember確認
3. 実行ClientのOperator確認
4. Mode文字列解析

エラー:

条件	Numeric
Parameter不足	461
Channelなし	403
Channel未参加	442
Operatorでない	482
未知Mode	472
対象Nickなし	401
対象がMemberでない	441

権限エラーの場合、Mode文字列を一切解析せず状態を変更しない。

16. MODE通知

通知先:

- ・ 変更後のChannel全Member
- ・ 実行Clientも含む

形式:

```
:<clientPrefix> MODE <channel> <changedModes> [changedParams]
```

Serverが自動Mode変更を行わないため、Prefixは常に実行Client Prefixである。

Mode変更後に送信バッファへqueueする。直接send()しない。

図の元データ:

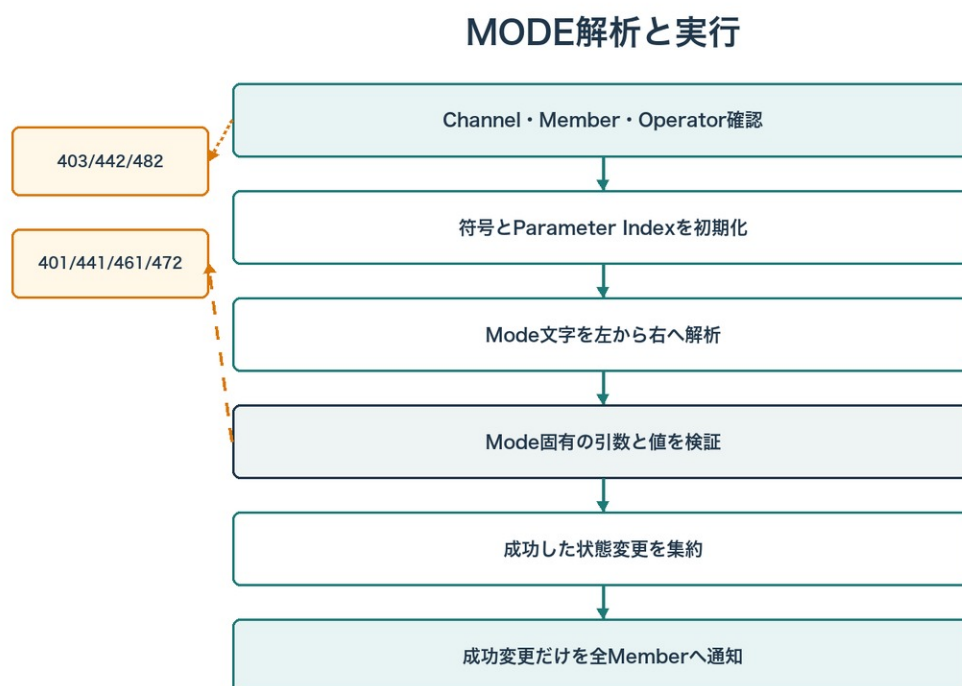


図1. MODE解析と実行 - 元データ: diagrams/mode_processing_detail_ja.mmd

17. 処理例

17.1 複数Flag

```
MODE #general +it
```

結果:

- ・ Invite-only有効
- ・ Topic制限有効
- ・ +itを全Memberへ通知

17.2 Parameter付き複数Mode

```
MODE #general +kol secret bob 10
```

左から:

1. +kがsecretを消費
2. +oがbobを消費
3. +lが10を消費

通知:

```
:alice!a@host MODE #general +kol secret bob 10
```

17.3 符号切替

```
MODE #general +it-k
```

結果:

- ・ +i
- ・ +t
- ・ -k

通知:

```
:alice!a@host MODE #general +it-k
```

17.4 一部失敗

```
MODE #general +io unknown
```

結果:

- ・ +i成功
- ・ +o unknownは401
- ・ Channelへ+iだけ通知

18. Modeと他Commandの依存

Mode	影響するCommand	判定場所
i	JOIN、INVITE	JOIN Handler
t	TOPIC	TOPIC Handler
k	JOIN	JOIN Handler
o	KICK、INVITE、TOPIC、MODE	共通権限確認
l	JOIN	JOIN Handler

Channelは状態を返すだけとし、Numeric ReplyやCommand判定を行わない。

19. テスト観点

- ・ MODE #channelで324が返る
- ・ +iと-iがJOINへ反映される
- ・ +tと-tがTOPICへ反映される
- ・ +kのKey一致と不一致を判定できる
- ・ -kが引数なしで動作する
- ・ +oと-oがOperator集合を更新する
- ・ +lが正の整数だけを受け付ける
- ・ -lが引数なしで動作する
- ・ 複数ModeのParameter消費順が正しい
- ・ 符号切替が通知文字列へ正しく反映される
- ・ 未知Modeと成功Modeを同時に処理できる
- ・ 一部Mode失敗後も後続Modeを処理できる
- ・ 実変更がないModeを通知しない
- ・ 非Operatorが変更できない

20. 実装完了条件

- ・ i、t、k、o、lの追加と削除がすべて実装されている
- ・ Mode照会が現在状態とParameterを返す
- ・ Mode文字列を左から右へ解析できる
- ・ Parameter消費規則がModeごとに正しい
- ・ 成功変更だけが1つの通知へ集約される
- ・ 一部エラーで成功済み変更が失われない
- ・ Channel状態とJOIN、TOPIC、Operator Commandの挙動が一致する
- ・ 未知Modeや不正Limitでサーバーがクラッシュしない